StudentSense Codebase Overview

StudentSense is a web application that helps students complete check-ins and find support, while giving counselors tools to follow up. The project is built with Django, a Python framework for websites.

Main folders

Folder	What it is used for
`studentsense/`	The website's main configuration and entry points. It connects the feature areas, sets up the database and other settings, and serves the site's shared pages.
`accounts/`	User accounts and profiles, including sign-in, registration, and notifications.
`intake/`	Student check-in forms, saved responses, results, and the code that uses the support-assessment model.
`intake/model_artifacts/`	The trained model and its details used by the intake feature to produce assessment results.
`recommendations/`	The pages and logic that provide support recommendations.
`counselor/`	Counselor tools for reviewing student support needs and managing related follow-up. It also contains a scheduled command for a weekly elevated-support digest.
`templates/`	The HTML pages shown in a web browser. Pages are grouped into subfolders such as `accounts/`, `intake/`, and `counselor/`.
`static/`	Files used by the website's appearance and browser features, including CSS, the logo, and the offline-support files.
`test/`	Automated checks for pages, forms, accounts, intake, counselor features, and performance.
`model_prototype/`	Scripts for experimenting with and training assessment models. These support model development and are separate from the model artifact used by the live intake feature.

Each feature area (`accounts/`, `intake/`, `recommendations/`, and `counselor/`) has Python files that handle its pages, web addresses, and data. Where needed, it also has forms and admin setup. Its `migrations/` folder records changes to the database structure over time.

Important files

File	What it is used for
`manage.py`	The command-line entry point for running the website and common Django tasks.
`studentsense/settings.py`	Sets the website's configuration, including installed features, database, templates, and static files.
`studentsense/urls.py`	Connects website addresses to the correct feature areas and pages.
`studentsense/views.py`	Handles shared pages such as the home, about, and offline pages.
`intake/model_service.py`	Loads and runs the trained assessment model for the intake feature.
`static/styles.css`	Provides the website's shared visual styling.
`static/sw.js`	Helps the website support offline behavior in compatible browsers.
`static/manifest.json`	Provides the browser with information used when installing the website like an app.
`requirements.txt`	Lists the Python packages needed by the main application.
`requirements-dev.txt`	Adds packages used for development and testing.
`requirements-pythonanywhere.txt`	Lists a smaller set of packages for a quota-limited PythonAnywhere deployment.
`db.sqlite3`	A local SQLite database file used by the current settings. It is local runtime data and is not included in the repository.

What this guide leaves out

This guide focuses on the parts that run the application or support its development. It does not list planning documents, editor settings, Python cache files, the local virtual environment, or generated static-file copies.